

Programming Best Practices: An Executive Summary for Data Analytics

Shruti Malik

ANLT 5010: Data Analytics

Week 7 Assignment

August 30, 2026

Kyle Camac

Programming Best Practices: An Executive Summary for Data Analytics

Data analytics programming has become an essential competency across industries that rely on evidence-based decision making. Writing code that is accurate, maintainable, and reproducible requires adherence to established best practices. This executive summary synthesizes key best practices in three critical areas: testing written programs, submitting and troubleshooting programs, and documentation and communication. Together, these practices ensure that analytics code is reliable, transparent, and useful to both technical and non-technical stakeholders.

Best Practices for Testing Programs

Rigorous testing is the foundation of trustworthy data analytics code. The primary goal of testing is to detect errors before they propagate into analysis results or business decisions. Software testing experts recommend a layered approach that includes unit testing, integration testing, and end-to-end validation (Dooley, 2017). Unit tests verify the correctness of individual functions or modules in isolation, while integration tests confirm that those components interact as intended when combined.

Automated testing frameworks such as pytest (Python) or testthat (R) enable analysts to run hundreds of tests in seconds, catching regressions each time code is modified (Percival & Gregory, 2020). Complementing automated tests, data validation checks should be built into pipelines to confirm that input datasets conform to expected schemas, value ranges, and completeness criteria before processing begins. McKinney (2022) emphasizes that "data quality issues caught at ingestion are exponentially cheaper to fix than those discovered after analysis is complete" (p. 47). Analysts should also apply the practice of test-driven development (TDD), writing test cases before writing the corresponding analytical code, to promote clear specification of expected behavior from the outset. These tests can be integrated into Continuous Integration/Continuous Deployment (CI/CD) pipelines, such as GitHub Actions, to automatically run checks upon every code submission.

Best Practices for Submitting and Troubleshooting Programs

Effective submission practices begin with version control. Using a system such as Git ensures that every change to a program is tracked, attributed, and reversible (Loeliger & McCullough, 2012). Before submitting any analytical program, analysts should run the full script in a clean environment to confirm that it executes without errors and produces the expected outputs. Submissions should include all

dependency information, such as a requirements.txt or environment.yml file, so that reviewers or collaborators can reproduce the exact computational environment.

Troubleshooting requires a systematic, hypothesis-driven approach. When errors arise, analysts should first read error messages carefully and trace them to the specific line and module where the failure occurred. Inserting print statements or using an interactive debugger (e.g., pdb in Python) to inspect variable states at key checkpoints is an efficient diagnostic strategy (Percival & Gregory, 2020). Logging, rather than ad hoc print statements, is preferable in production pipelines because it provides a persistent record of execution events and error conditions. When a bug cannot be isolated quickly, isolating the minimal reproducible example (MRE) accelerates resolution by removing irrelevant code and data, allowing the analyst to focus precisely on the faulty logic. Furthermore, utilizing static code analysis and linting tools, such as flake8 or pylint in Python, helps analysts catch syntax and style errors before the code is even run, significantly reducing troubleshooting time.

Best Practices for Documentation and Communication

Documentation is the bridge between analytical code and its audience. At the code level, inline comments should explain why a particular approach was chosen, not merely what the code does, as the latter is already evident from the syntax itself (Dooley, 2017). Functions and classes should include docstrings that describe parameters, return values, and any exceptions raised. For data analytics specifically, documenting data lineage—the origin, transformations applied, and destination of each dataset—is critical for audit trails and regulatory compliance.

At the project level, a README file should provide an overview of the program's purpose, installation instructions, usage examples, and known limitations. McKinney (2022) recommends that analytical notebooks include narrative text alongside code cells so that readers can follow the analytical reasoning without needing to interpret every line of code. Communication to non-technical stakeholders should translate technical findings into plain language, emphasizing business implications rather than implementation details. Visualizations, summaries, and executive-level reports should be clearly labeled and accompanied by concise interpretive text that guides readers to actionable conclusions.

Conclusion

Adhering to best practices for testing, submission, troubleshooting, and documentation elevates data analytics programming from a technical exercise to a professional discipline. Testing ensures that

programs produce correct results under varied conditions; structured submission and troubleshooting workflows minimize downtime and errors in production; and thorough documentation enables collaboration, reproducibility, and communication with diverse audiences. Organizations that invest in these practices reduce technical debt, accelerate onboarding, and increase confidence in the data-driven insights they produce.

References

- Dooley, J. F. (2017). *Software development and professional practice*. Apress.
<https://doi.org/10.1007/978-1-4302-4843-6>
- Loeliger, J., & McCullough, M. (2012). *Version control with Git: Powerful tools and techniques for collaborative software development (2nd ed.)*. O'Reilly Media.
- McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3rd ed.)*. O'Reilly Media.
- Percival, H., & Gregory, B. (2020). *Architecture patterns with Python: Enabling test-driven development, domain-driven design, and event-driven microservices*. O'Reilly Media.